消息队列面试题

大家好，我是小林。

消息队列这块，在面试里出现的频率越来越高，尤其是做过有一定规模业务的同学，基本逃不掉。很多人平时用 MQ 就是往里发消息、消费消息，问起来能说出个"解耦、异步、削峰"，但一旦被追问"消息丢了怎么办""消息重复了怎么处理""积压了几百万条消息怎么解决"，就开始说不清楚了。这块知识其实不算难，但要答得有深度，还是得把原理搞透。

这篇文章整理了消息队列面试中最常被问到的知识点，涵盖 MQ 的使用场景、消息可靠性、幂等处理、消息积压，以及 RocketMQ、Kafka、RabbitMQ 三大主流 MQ 的核心原理和对比，覆盖比较全面。

有几块在面试里特别容易被深挖，建议重点花时间：

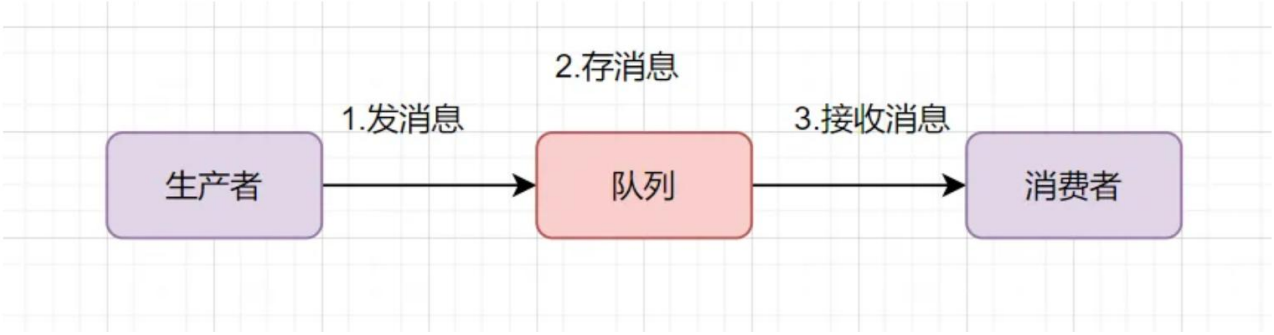
- **消息可靠性**：生产者、Broker、消费者三个环节各自怎么保证不丢消息，这是必考题，而且考官很喜欢追问每个环节的细节。
- **重复消费和幂等**：为什么会重复、怎么设计幂等方案，这个和业务结合很紧，答起来容易流于表面。
- **消息积压**：如何快速处理积压，尤其是 RocketMQ 里队列数和消费者数的关系，是很多人的盲区。
- **Kafka 和 RocketMQ 的区别**：存储模型、协调机制、适用场景，这是高频对比题，面试里问得很多。
- **事务消息**：怎么保证本地事务和消息发送的一致性，半消息机制是怎么工作的，这块能和分布式事务挂钩，答好了很加分。

如果你是第一次系统准备这块，建议先搞清楚 MQ 的基本原理（为什么用、怎么用），再去看可靠性和幂等这两个核心问题，然后再对比几个主流 MQ 的差异，这个顺序学下来会比较顺。

消息队列场景

什么是消息队列?

你可以把消息队列理解为一个**使用队列来通信**的组件。它的本质，就是个**转发器**，包含**发消息、存消息、消费消息**的过程。最简单的消息队列模型如下：



我们通常说的消息队列，简称 MQ（Message Queue），它其实就指**消息中间件**，当前业界比较流行的开源消息中间件包括：`RabbitMQ、RocketMQ、Kafka`。

为什么需要消息队列?

如果用一句话概括为什么我们要大费周章地引入 MQ，核心其实就是为了解决系统架构中的三大痛点：**解耦、异步、削峰**。

我可以结合实际的业务场景，给你连贯地汇报一下这三点：

第一点，最核心的作用是为了「解耦」（把绑在一起的系统拆开）。 在没有 MQ 的时候，假设我们做个电商系统，用户付完款后，系统要干好几件事：扣库存、发优惠券、加积分、还要发短信通知。如果订单系统直接用 RPC 调用去挨个通知这些子系统，那代码就高度耦合在了一起。万一有一天不需要发短信了，或者新加了一个自动发货服务，订单系统的研发就得整天去改代码重新发版；更别提如果哪天「发积分服务」挂了报错，甚至会连累整个订单支付流程跟着失败。 引入 MQ 后，整个逻辑就变了。订单系统付完款，它只需要对着 MQ 丢一句话（发一条消息）：「用户 A 支付成功了」，它的主线任务就彻底结束了。后续不管有几个子系统关心这件事，它们自己去订阅 MQ 就好了，哪怕以后新加再多依赖服务，订单系统的代码也一行都不用改。这就达到了物理和逻辑上的彻底解耦，系统容错率极高。

第二点，顺理成章带来的性能提升叫作「异步」。 紧接上面的例子，如果要串行调用发短信、加积分等等这么多非核心流程，哪怕每个接口只用 50 毫秒，加起来用户在页面等待的时间可能就得大半秒甚至好几秒，疯狂转圈圈，体验极差。 有了 MQ，主流程其实只需要把状态改掉，然后往 MQ 里抛一条消息（耗时大概几毫秒），立刻就可以给前端响应「恭喜你支付成功」了。像发短信、加积分这种「不是非得在这零点几秒内完成」的操作，完全可以让系统在后台顺着 MQ 慢慢执行，这样前台接口的响应时间就大幅缩短了。

第三点，是高并发场景下的保命神器，叫作「削峰填谷」。 平常我们底层的 MySQL 数据库，一秒钟扛个一两千并发可能没事。但是一旦搞促销、秒杀，瞬间如果有两万个请求涌进来，如果直接打到数据库上，数据库绝对当场宕机。 这个时候，MQ 就扮演了一个极其关键的**「蓄水池」或者

「排队区」**的作用。这两万个瞬时狂暴请求过来，我们统一先扔进 MQ 里暂存着。然后后端的订单处理程序，根据自己数据库真实的抗压能力，平稳地、以一秒钟一两千个的速度，慢慢从 MQ 里往外拉任务来消化。这就像游乐场门口的蛇形排队通道，把瞬间爆发的流量洪峰（削峰），分散到了后续慢慢处理（填谷）。虽然用户体感上可能是排队多了几秒钟，但总比整个大盘系统崩溃要好无数倍。

总结一下： 引入消息队列，确实会让系统整体变复杂，还需要处理消息不丢失、不重复消费这些麻烦事儿。但是为了换取各个服务之间的**解耦**、核心接口通过**异步**大幅减少响应时间、以及面对几倍几十倍突发流量时的**削峰**保命，这在稍微复杂一点的系统里都是极其必要的。这就是我们需要 MQ 的根本原因。

消息队列有什么缺点?

关于消息队列（MQ）的优缺点，在软件架构设计里有一句老话叫「没有银弹」。引入 MQ 绝对是一次典型的 Trade-off（技术权衡），它既能拯救系统，又能带来一堆让人头疼的麻烦。

我们可以分两面来看：

首先是它的优点，总结起来就是「系统架构救命的三板斧」：

1. **解耦**：把各个原本互相关联的系统拆解开。A 系统只要把消息丢给 MQ 就完事了，B 系统或者日后新加的 C 系统想要数据，自己去 MQ 拿就行。A 系统的代码不用再为了别的系统频繁修改，大家各干各的。
2. **异步**：提升了用户的响应速度。一些非核心、但又很费时间的操作（比如发短信、送积分），全部扔进 MQ 后台慢慢执行。主流程几毫秒就结束了，用户的体感极佳。
3. **削峰**：这是大促、秒杀时挡在数据库前面的超级盾牌。把洪水猛兽般瞬间涌入的并发请求，先拦截在 MQ 的队列里，变成了细水长流，让数据库能按照自己的节奏平稳处理，保住系统不崩溃。

但是，既然它这么好，为什么不每个地方都加呢？这就不得不提它带来的三大缺点或者说是巨大挑战：

第一，系统整体的「可用性」反而变低了。 在没有 MQ 时，A 系统直接调 B 系统，只要它俩不出问题就行。现在中间横插了一个 MQ 组件，多拉了一个「中间人」。一旦这个机器宕机、网络故障，整个业务链路直接瘫痪。所以引入 MQ，你就不得不花大价钱和精力去搭比如主从集群，去保证 MQ 的绝对高可用，不然这就是给系统埋下了一颗大雷。

第二，系统的「开发复杂性」呈指数级上升。 原本很简单的一个方法调用，换成发消息之后，开发人员就面临无数个让人掉头发的异常场景。比如：网络抖动导致一条消息发了两次，B 系统怎么保证不把老王的钱扣两次？（这就需要做跨业务的**防重和幂等性设计**）；再比如，MQ 自己重启了，还没处理完的消息怎么保证不丢失？这些都需要我们在写代码时增加非常多的确认机制、补偿机制。

第三，最头疼的「数据一致性」问题。 A 系统本地业务执行成功了，消息发给 MQ 了，就高高兴兴告诉前端「操作成功」。结果下游的 B 系统拿到这条消息后，由于一条空指针异常没处理好，业务失败了。现在好了，A 以为成功了，B 实际失败了，两边的数据产生彻底的脱节。想要解决这个问题，我们就不得不引入相对复杂的「分布式事务」方案，或者开发额外的对账及人工补偿工具。

总结一下： MQ 就像是一把威力巨大的重型武器，优点极其耀眼。但它的缺点决定了，我们绝不是逢坑就用。只有当业务场景真的对解耦、异步、抗高并发有强烈需求，且这部分收益远远大过引入这些复杂性带来的麻烦时，我们才会去谨慎地使用它。

消息队列怎么选型?

Kafka、ActiveMQ、RabbitMQ、RocketMQ 来进行不同维度对比。

特性	ActiveMQ	RabbitMQ	RocketMQ	Kafka
单机吞吐量	万级	万级	10 万级	10 万级
时效性	毫秒级	微秒级	毫秒级	毫秒级
可用性	高（主从）	高（主从）	非常高（分布式）	非常高（分布式）
消息重复	至少一次	至少一次	至少一次 最多一次	至少一次最多一次
消息顺序性	有序	有序	有序	分区有序
支持主题数	千级	百万级	千级	百级，多了性能严重下滑

特性	ActiveMQ	RabbitMQ	RocketMQ	Kafka
消息回溯	不支持	不支持	支持（按时间回溯）	支持（按 offset 回溯）
管理界面	普通	普通	完善	普通

选型的时候，我们需要根据业务场景，结合上述特性来进行选型。

比如你要支持天猫双十一类超大型的秒杀活动，这种一锤子买卖，那管理界面、消息回溯啥的不重要。

我们需要看什么？看吞吐量！

所以优先选 Kafka 和 RocketMQ 这种更高吞吐的。

比如做一个公司的中台，对外提供能力，那可能会有很多主题接入，这时候主题个数又是很重要的考量，像 Kafka 这样百级的，就不太符合要求，可以根据情况考虑千级的 RocketMQ，甚至百万级的 RabbitMQ。

又比如是一个金融类业务，那么重点考虑的就是稳定性、安全性，分布式部署的 Kafka 和 Rocket 就更有优势。

特别说一下时效性，RabbitMQ 以微秒的时效作为招牌，但实际上毫秒和微秒，在绝大多数情况下，都没有感知的区别，加上网络带来的波动，这一点在生产过程中，反而不会作为重要的考量。

其它的特性，如消息确认、消息回溯，也经常作为考量的场景，管理界面的话试公司而定了，反正我呆过的地方，都不看重这个，毕竟都有自己的运维体系。

消息队列使用场景有哪些?

- **解耦**：可以在多个系统之间进行解耦，将原本通过网络之间的调用的方式改为使用 MQ 进行消息的异步通讯，只要该操作不是需要同步的，就可以改为使用 MQ 进行不同系统之间的联系，这样项目之间不会存在耦合，系统之间不会产生太大的影响，就算一个系统挂了，也只是消息挤压在 MQ 里面没人进行消费而已，不会对其他的系统产生影响。
- **异步**：加入一个操作设计到好几个步骤，这些步骤之间不需要同步完成，比如客户去创建了一个订单，还要去客户轨迹系统添加一条轨迹、去库存系统更新库存、去客户系统修改客户的状态等等。这样如果这个系统都直接进行调用，那么将会产生大量的时间，这样对于客户

是无法接收的；并且像添加客户轨迹这种操作是不需要去同步操作的，如果使用 MQ 将客户创建订单时，将后面的轨迹、库存、状态等信息的更新全都放到 MQ 里面然后去异步操作，这样就可加快系统的访问速度，提供更好的客户体验。

- **削峰**：一个系统访问流量有高峰时期，也有低峰时期，比如说，中午整点有一个抢购活动等等。比如系统平时流量并不高，一秒钟只有 100 多个并发请求，系统处理没有任何压力，一切风平浪静，到了某个抢购活动时间，系统并发访问了剧增，比如达到了每秒 5000 个并发请求，而我们的系统每秒只能处理 2000 个请求，那么由于流量太大，我们的系统、数据库可能就会崩溃。这时如果使用 MQ 进行流量削峰，将用户的大量消息直接放到 MQ 里面，然后我们的系统去按自己的最大消费能力去消费这些消息，就可以保证系统的稳定，只是可能要跟进业务逻辑，给用户返回特定页面或者稍后通过其他方式通知其结果

消息重复消费怎么解决?

生产端为了保证消息发送成功，可能会重复推送(直到收到成功 ACK)，会产生重复消息。但是一个成熟的 MQ Server 框架一般会想办法解决，避免存储重复消息(比如：空间换时间，存储已处理过的 message_id)，给生产端提供一个幂等性的发送消息接口。

但是消费端却无法根本解决这个问题，在高并发标准要求下，拉取消息+业务处理+提交消费位移需要做事务处理，另外消费端服务可能宕机，很可能会拉取到重复消息。

所以，只能业务端自己做控制，**对于已经消费成功的消息，本地数据库表或 Redis 缓存业务标识，每次处理前先进行校验，保证幂等。**

消息丢失怎么解决的?

使用一个消息队列，其实就分为三大块：**生产者、中间件、消费者**，所以要保证消息就是保证三个环节都不能丢失数据。



- **消息生产阶段**：生产者会不会丢消息，取决于生产者对于异常情况的处理是否合理。从消息被生产出来，然后提交给 MQ 的过程中，只要能正常收到 （ MQ 中间件） 的 ack 确认响

应，就表示发送成功，所以只要处理好返回值和异常，如果返回异常则进行消息重发，那么这个阶段是不会出现消息丢失的。

- **消息存储阶段**：Kafka 在使用时是部署一个集群，生产者在发布消息时，队列中间件通常会写「多个节点」，也就是有多个副本，这样一来，即便其中一个节点挂了，也能保证集群的数据不丢失。
- **消息消费阶段**：消费者接收消息+消息处理之后，才回复 ack 的话，那么消息阶段的消息不会丢失。不能收到消息就回 ack，否则可能消息处理中途挂掉了，消息就丢失了。

使用消息队列还应该注意哪些问题?

需要考虑消息可靠性和顺序性方面的问题。

消息队列的可靠性、顺序性怎么保证?

消息可靠性可以通过下面这些方式来保证

- **消息持久化**：确保消息队列能够持久化消息是非常关键的。在系统崩溃、重启或者网络故障等情况下，未处理的消息不应丢失。例如，像 RabbitMQ 可以通过配置将消息持久化到磁盘，通过将队列和消息都设置为持久化的方式（设置 `durable = true`），这样在服务器重启后，消息依然可以被重新读取和处理。
- **消息确认机制**：消费者在成功处理消息后，应该向消息队列发送确认（acknowledgment）。消息队列只有收到确认后，才会将消息从队列中移除。如果没有收到确认，消息队列可能会在一定时间后重新发送消息给其他消费者或者再次发送给同一个消费者。以 Kafka 为例，消费者通过 `commitSync` 或者 `commitAsync` 方法来提交偏移量（offset），从而确认消息的消费。
- **消息重试策略**：当消费者处理消息失败时，需要有合理的重试策略。可以设置重试次数和重试间隔时间。例如，在第一次处理失败后，等待一段时间（如 5 秒）后进行第二次重试，如果重试多次（如 3 次）后仍然失败，可以将消息发送到死信队列，以便后续人工排查或者采取其他特殊处理。

消息顺序性保证的方式如下：

- **有序消息处理场景识别**：首先需要明确业务场景中哪些消息是需要保证顺序的。例如，在金融交易系统中，对于同用户的转账操作顺序是不能打乱的。对于需要顺序处理的消息，要确保消息队列和消费者能够按照特定的顺序进行处理。
- **消息队列对顺序性的支持**：部分消息队列本身提供了顺序性保证的功能。比如 Kafka 可以通过将消息划分到同一个分区（Partition）来保证消息在分区内是有序的，消费者按照分区顺

序读取消息就可以保证消息顺序。但这也可能会限制消息的并行处理程度，需要在顺序性和吞吐量之间进行权衡。

- **消费者顺序处理策略**：消费者在处理顺序消息时，应该避免并发处理可能导致顺序打乱的情况。例如，可以通过单线程或者使用线程池并对顺序消息进行串行化处理等方式，确保消息按照正确的顺序被消费。

如何保证幂等写?

幂等性是指 **同一操作的多次执行对系统状态的影响与一次执行结果一致**。例如，支付接口若因网络重试被多次调用，最终应确保仅扣款一次。实现幂等写的核心方案：

- 唯一标识（幂等键）：客户端为每个请求生成全局唯一 ID（如 UUID、业务主键），服务端校验该 ID 是否已处理，适用场景接口调用、消息消费等。
- 数据库事务 + 乐观锁：通过版本号或状态字段控制并发更新，确保多次更新等同于单次操作，适用场景数据库记录更新（如余额扣减、订单状态变更）。
- 数据库唯一约束：利用数据库唯一索引防止重复数据写入，适用场景数据插入场景（如订单创建）。
- 分布式锁：通过锁机制保证同一时刻仅有一个请求执行关键操作，适用场景高并发下的资源抢夺（如秒杀）。
- 消息去重：消息队列生产者为每条消息生成唯一的消息 ID，消费者在处理消息前，先检查该消息 ID 是否已经处理过，如果已经处理过则丢弃该消息。

如何处理消息队列的消息积压问题?

消息积压是因为生产者的生产速度，大于消费者的消费速度。遇到消息积压问题时，我们需要先排查，是不是有 bug 产生了。

如果不是 bug，我们可以**优化一下消费的逻辑**，比如之前是一条一条消息消费处理的话，我们可以确认是不是可以优为**批量处理消息**。如果还是慢，我们可以考虑水平扩容，增加 Topic 的队列数，和消费组机器的数量，提升整体消费能力。

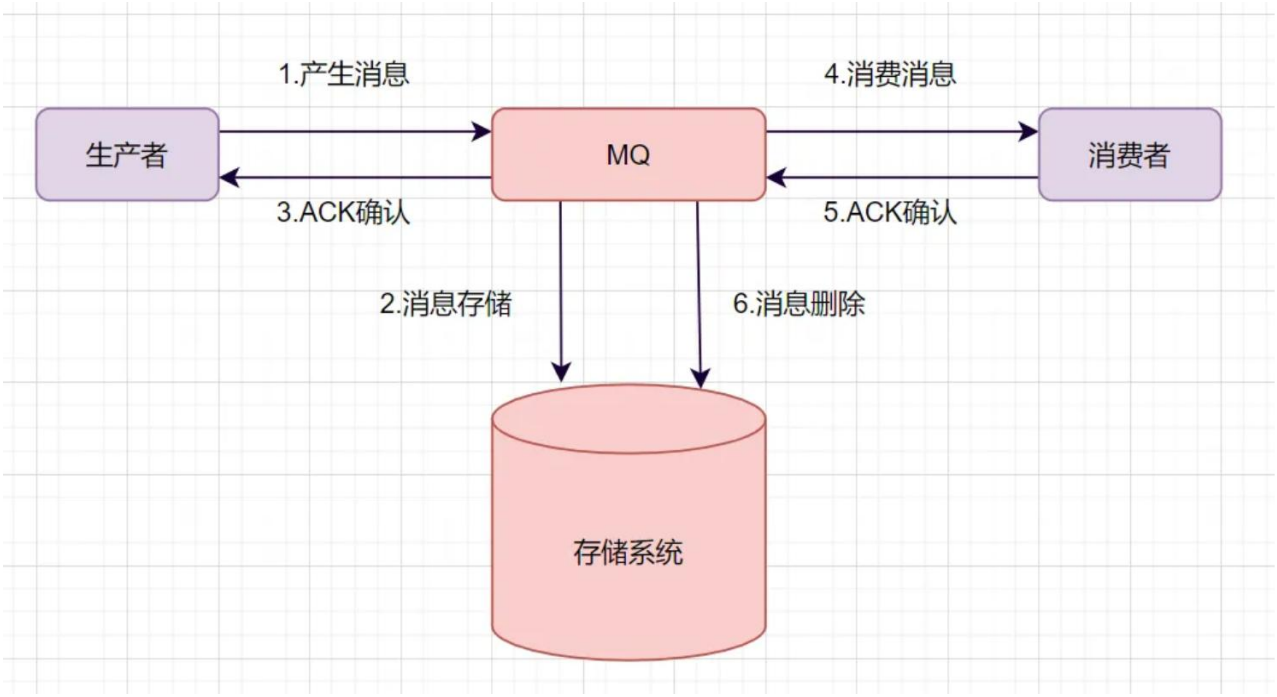
如果是 bug 导致几百万消息持续积压几小时。有如何处理呢？需要解决 bug，**临时紧急扩容**，大概思路如下：

> 1. 先修复 consumer 消费者的问题，以确保其恢复消费速度，然后将现有 consumer 都停掉。
> 2. 新建一个 topic，partition 是原来的 10 倍，临时建立好原先 10 倍的 queue 数量。

3. 然后写一个临时的分发数据的 consumer 程序，这个程序部署上去消费积压的数据，消费之后不做耗时的处理，直接均匀轮询写入临时建立好的 10 倍数量的 queue。
4. 接着临时征用 10 倍的机器来部署 consumer，每一批 consumer 消费一个临时 queue 的数据。这种做法相当于是临时将 queue 资源和 consumer 资源扩大 10 倍，以正常的 10 倍速度来消费数据。
5. 等快速消费完积压数据之后，得恢复原先部署的架构，重新用原先的 consumer 机器来消费消息。

如何保证数据一致性，事务消息如何实现？

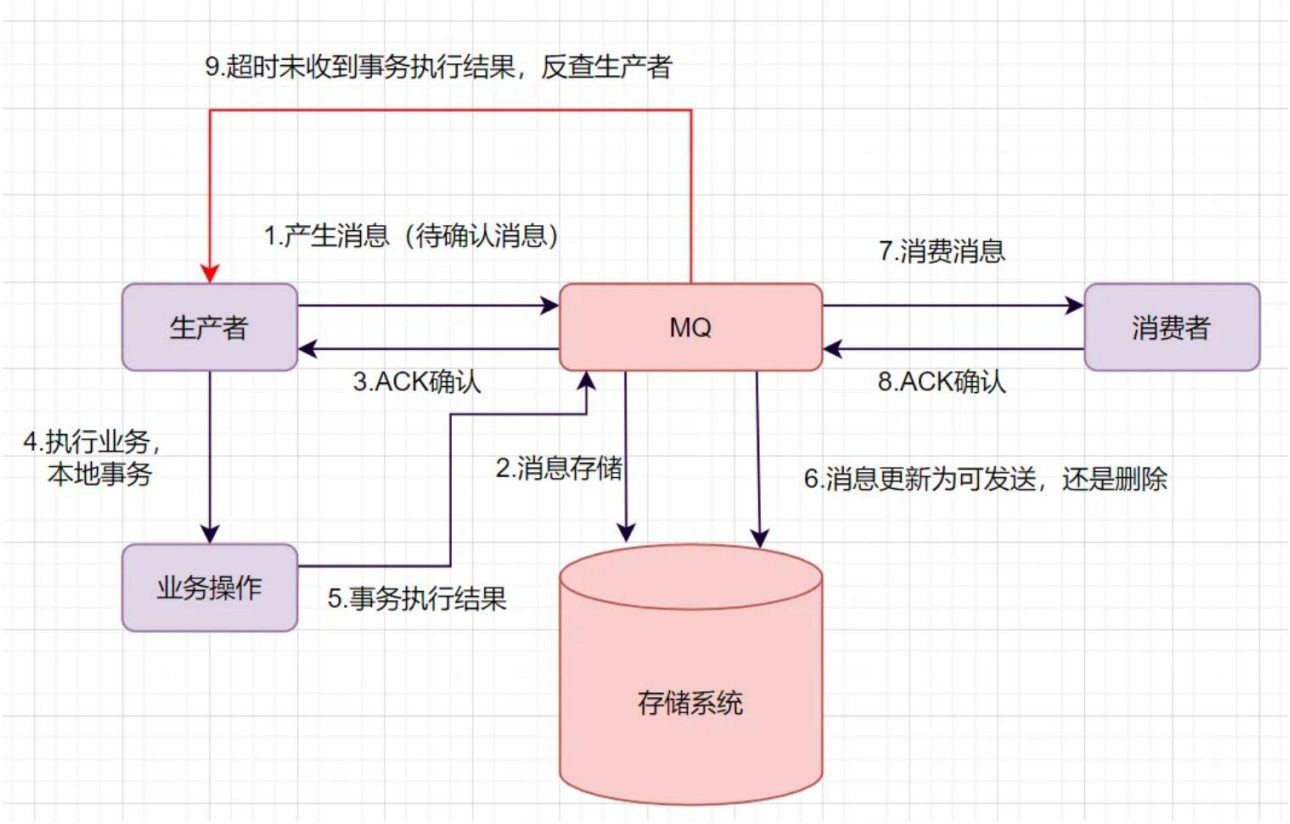
一条普通的 MQ 消息，从产生到被消费，大概流程如下：



1. 生产者产生消息，发送带 MQ 服务器
2. MQ 收到消息后，将消息持久化到存储系统。
3. MQ 服务器返回 ACk 到生产者。
4. MQ 服务器把消息 push 给消费者
5. 消费者消费完消息，响应 ACK
6. MQ 服务器收到 ACK，认为消息消费成功，即在存储中删除消息。

我们举个**下订单**的例子吧。订单系统创建完订单后，再发送消息给下游系统。如果订单创建成功，然后消息没有成功发送出去，下游系统就无法感知这个事情，出导致数据不一致。

如何保证数据一致性呢？可以使用**事务消息**。一起来看下事务消息是如何实现的吧。



1. 生产者产生消息，发送一条**半事务消息**到 MQ 服务器
2. MQ 收到消息后，将消息持久化到存储系统，这条消息的状态是**待发送**状态。
3. MQ 服务器返回 ACK 确认到生产者，此时 MQ 不会触发消息推送事件
4. 生产者执行本地事务
5. 如果本地事务执行成功，即 commit 执行结果到 MQ 服务器；如果执行失败，发送 rollback。
6. 如果是正常的 commit，MQ 服务器更新消息状态为**可发送**；如果是 rollback，即删除消息。
7. 如果消息状态更新为可发送，则 MQ 服务器会 push 消息给消费者。消费者消费完就回 ACK。
8. 如果 MQ 服务器长时间没有收到生产者的 commit 或者 rollback，它会反查生产者，然后根据查询到的结果执行最终状态。

消息队列是参考哪种设计模式?

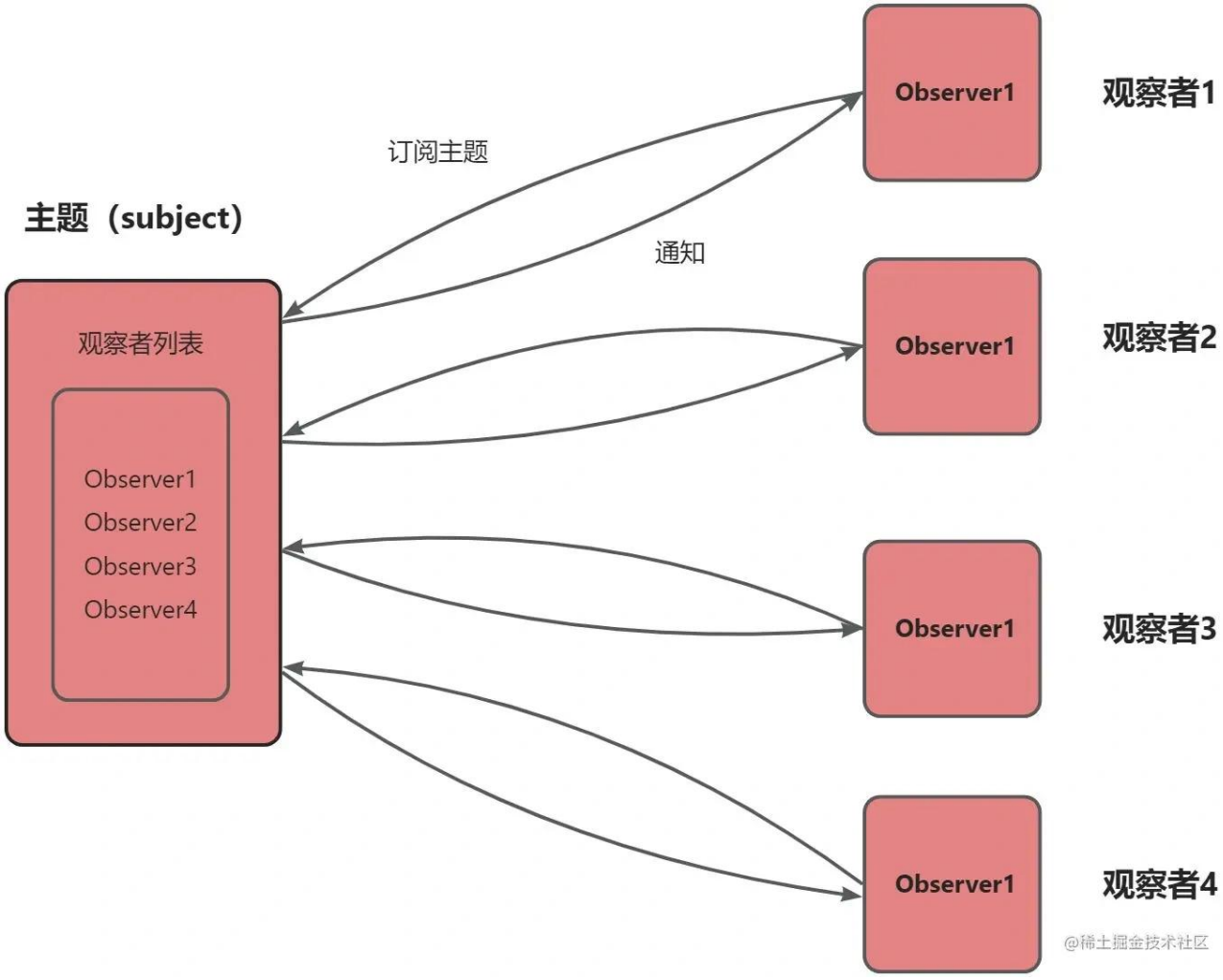
是参考了观察者模式和发布订阅模式，两种设计模式思路是一样的，举个生活例子：

- 观察者模式：某公司给自己员工发月饼发粽子，是由公司的行政部门发送的，这件事不适合交给第三方，原因是「公司」和「员工」是一个整体
- 发布-订阅模式：某公司要给其他人发各种快递，因为「公司」和「其他人」是独立的，其唯一的桥梁是「快递」，所以这件事适合交给第三方快递公司解决

上述过程中，如果公司自己去管理快递的配送，那公司就会变成一个快递公司，业务繁杂难以管理，影响公司自身的主营业务，因此使用何种模式需要考虑什么情况两者是需要耦合的

> 观察者模式

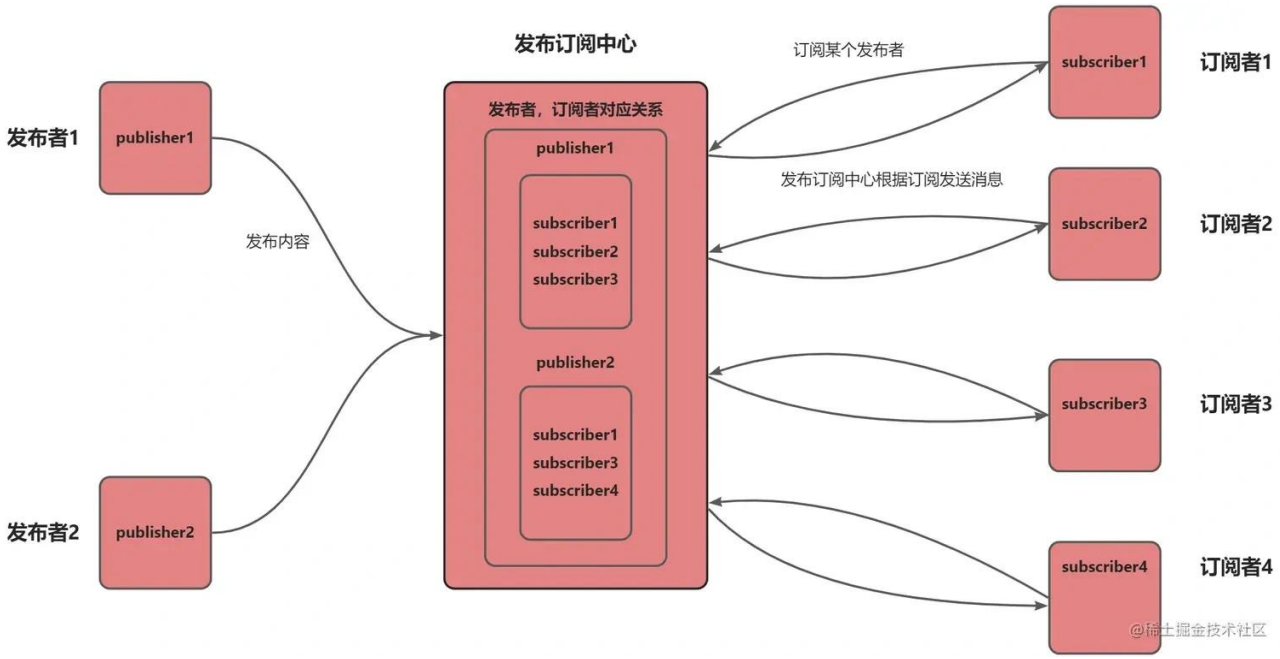
观察者模式实际上就是一个一对多的关系，在观察者模式中存在一个主题和多个观察者，主题也是被观察者，当我们主题发布消息时，会通知各个观察者，观察者将会收到最新消息，图解如下：每个观察者首先订阅主题，订阅成功后当主题发送消息时会循环整个观察者列表，逐一发送消息通知。



> 发布订阅模式

发布订阅模式和观察者模式的区别就是发布者和订阅者完全解耦，通过中间的发布订阅中心进行消息通知，发布者并不知道自己发布的消息会通知给谁，因此发布订阅模式有三个重要角色，发布者->发布订阅中心->订阅者。

图解如下：当发布者发布消息到发布订阅中心后，发布订阅中心会将消息通知给所有订阅该发布者的订阅者



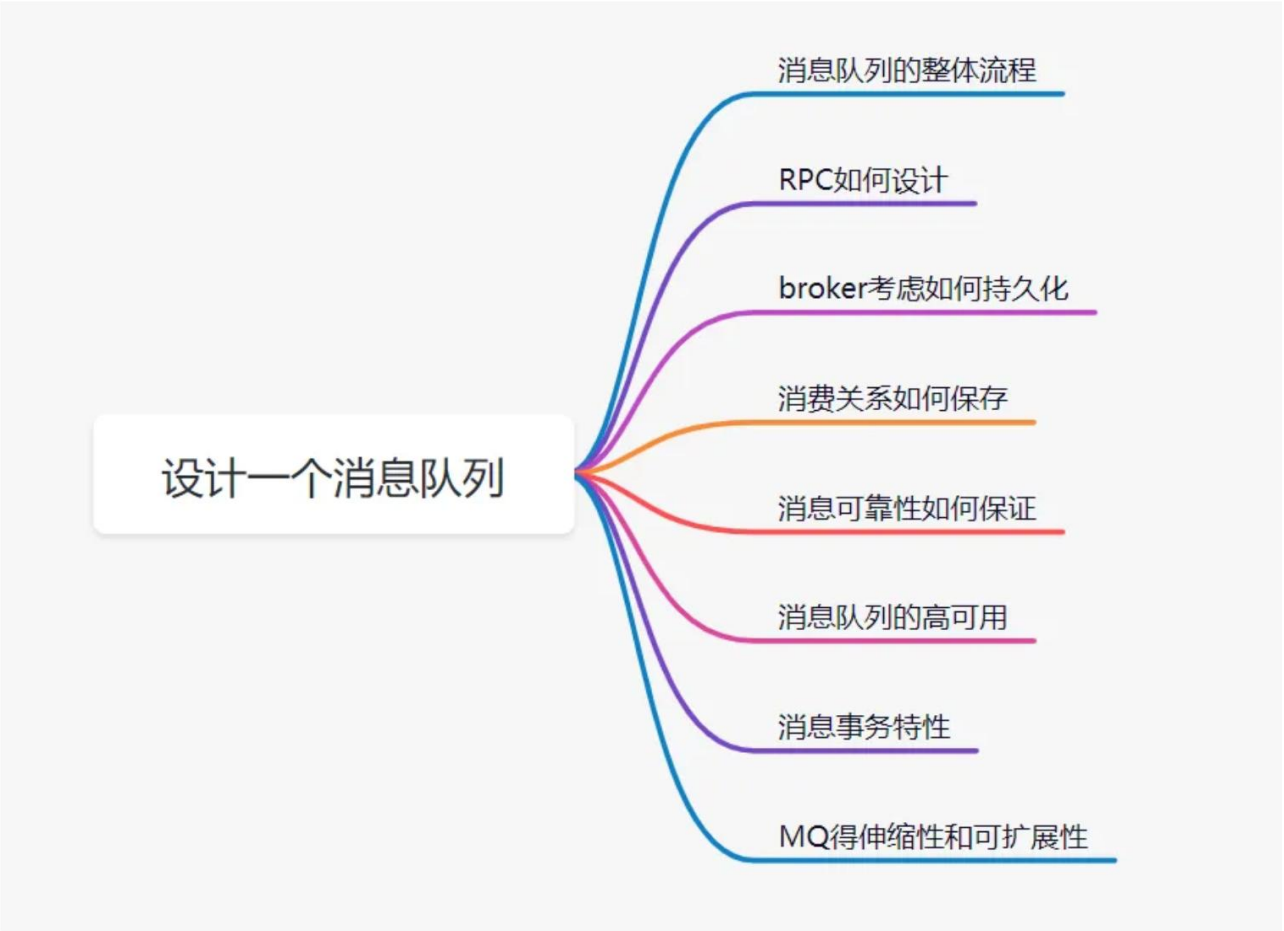
让你写一个消息队列，该如何进行架构设计?

这个问题面试官主要考察三个方面的知识点：

- 你有没有对消息队列的架构原理比较了解
- 考察你的个人设计能力
- 考察编程思想，如什么高可用、可扩展性、幂等等等。

遇到这种设计题，大部分人会很蒙圈，因为平时没有思考过类似的问题。大多数人平时埋头增删改啥，不去思考框架背后的一些原理。有很多类似的问题，比如让你来设计一个 Dubbo 框架，或者让你来设计一个 MyBatis 框架，你会怎么思考呢？

回答这类问题，并不要求你研究过那技术的源码，你知道那个技术框架的基本结构、工作原理即可。设计一个消息队列，我们可以从这几个角度去思考：



1. 首先是消息队列的整体流程，producer 发送消息给 broker，broker 存储好，broker 再发送给 consumer 消费，consumer 回复消费确认等。
2. producer 发送消息给 broker，broker 发消息给 consumer 消费，那就需要两次 RPC 了，RPC 如何设计呢？可以参考开源框架 Dubbo，你可以说说服务发现、序列化协议等等
3. broker 考虑如何持久化呢，是放文件系统还是数据库呢，会不会消息堆积呢，消息堆积如何处理呢。
4. 消费关系如何保存呢？点对点还是广播方式呢？广播关系又是如何维护呢？zk 还是 config server
5. 消息可靠性如何保证呢？如果消息重复了，如何幂等处理呢？
6. 消息队列的高可用如何设计呢？可以参考 Kafka 的高可用保障机制。多副本 -> leader & follower -> broker 挂了重新选举 leader 即可对外服务。
7. 消息事务特性，与本地业务同个事务，本地消息落库;消息投递到服务端，本地才删除；定时任务扫描本地消息库，补偿发送。
8. MQ 得伸缩性和可扩展性，如果消息积压或者资源不够时，如何支持快速扩容，提高吞吐？可以参照一下 Kafka 的设计理念，broker -> topic -> partition，每个 partition 放一个

机器，就存一部分数据。如果现在资源不够了，简单啊，给 topic 增加 partition，然后做数据迁移，增加机器，不就可以存放更多数据，提供更高的吞吐量了。

RocketMQ

消息队列为什么选择 RocketMQ 的?

项目用的是 RocketMQ 消息队列。选择 RocketMQ 的原因是：

- **开发语言优势**。RocketMQ 使用 Java 语言开发，比起使用 Erlang 开发的 RabbitMQ 来说，有着更容易上手的阅读体验和受众。在遇到 RocketMQ 较为底层的问题时，大部分熟悉 Java 的同学都可以深入阅读其源码，分析、排查问题。
- **社区氛围活跃**。RocketMQ 是阿里巴巴开源且内部在大量使用的消息队列，说明 RocketMQ 是的确经得起残酷的生产环境考验的，并且能够针对线上环境复杂的需求场景提供相应的解决方案。
- **特性丰富**。根据 RocketMQ 官方文档的列举，其高级特性达到了 `12 种`，例如顺序消息、事务消息、消息过滤、定时消息等。顺序消息、事务消息、消息过滤、定时消息。RocketMQ 丰富的特性，能够为我们在复杂的业务场景下尽可能多地提供思路及解决方案。

RocketMQ 和 Kafka 的区别是什么？如何做技术选型?

Kafka 的优缺点：

- 优点：首先，Kafka 的最大优势就在于它的高吞吐量，在普通机器 4CPU8G 的配置下，一台机器可以抗住十几万的 QPS，这一点还是相当优越的。Kafka 支持集群部署，如果部分机器宕机不可用，则不影响 Kafka 的正常使用。
- 缺点：Kafka 有可能会造成数据丢失，因为它在收到消息的时候，并不是直接写到物理磁盘的，而是先写入到磁盘缓冲区里面的。Kafka 功能比较的单一 主要的就是支持收发消息，高级功能基本没有，就会造成适用场景受限。

RocketMQ 是阿里巴巴开源的消息中间件，优缺点

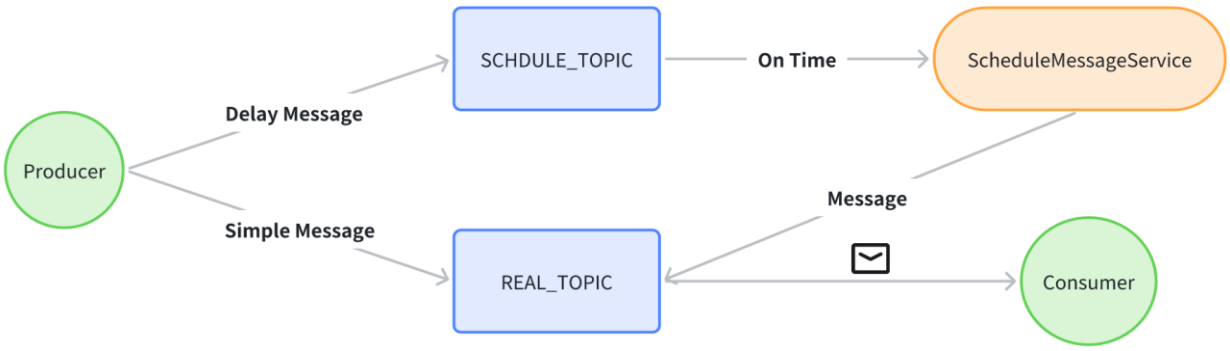
- 优点：支持功能比较多，比如延迟队列、消息事务等等，吞吐量也高，单机吞吐量达到 10 万级，支持大规模集群部署，线性扩展方便，Java 语言开发，满足了国内绝大部分公司技术栈
- 缺点：性能相比 kafka 是弱一点，因为 kafka 用到了 sendfile 的零拷贝技术，而 RocketMQ 主要是用 mmap+write 来实现零拷贝。

该怎么选择呢？

- 如果我们业务只是收发消息这种单一类型的需求，而且可以允许小部分数据丢失的可能性，但是又要求极高的吞吐量和高性能的话，就直接选 Kafka 就行了，就好比我们公司想要收集和传输用户行为日志以及其他相关日志的处理，就选用的 Kafka 中间件。
- 如果公司的需要通过 mq 来实现一些业务需求，比如延迟队列、消息事务等，公司技术栈主要是 Java 语言的话，就直接一步到位选择 RocketMQ，这样会省很多事情。

RocketMQ 延时消息的底层原理

总体的原理示意图，如下所示：



broker 在接收到延时消息的时候，会将延时消息存入到延时 Topic 的队列中，然后 ScheduleMessageService 中，每个 queue 对应的定时任务会不停地被执行，检查 queue 中哪些消息已到设定时间，然后转发到消息的原始 Topic，这些消息就会被各自的 producer 消费了。

RocektMQ 怎么处理分布式事务?

RocketMQ 是一种最终一致性的分布式事务，就是说它保证的是消息最终一致性，而不是像 2PC、3PC、TCC 那样强一致分布式事务

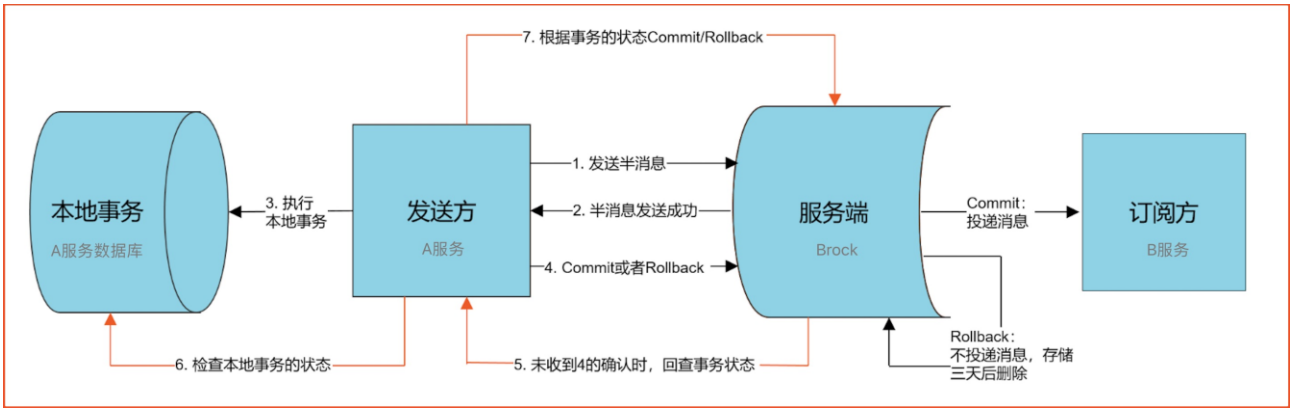
假设 A 给 B 转 **100 块钱**，同时它们不是同一个服务上，现在目标是就是 A 减 100 块钱，B 加 100 块钱。

实际情况可能有四种：

- 1）就是 A 账户减 100 （成功），B 账户加 100 （成功）
- 2）就是 A 账户减 100（失败），B 账户加 100 （失败）
- 3）就是 A 账户减 100（成功），B 账户加 100 （失败）
- 4）就是 A 账户减 100 （失败），B 账户加 100 （成功）

这里 **第 1 和第 2** 种情况是能够保证事务的一致性的，但是 **第 3 和第 4** 是无法保证事务的一致性的。

那我们来看下 RocketMQ 是如何来保证事务的一致性的。



分布式事务的流程如上图：

- 1、A 服务先发送个 Half Message（是指暂不能被 Consumer 消费的消息。Producer 已经把消息成功发送到了 Broker 端，但此消息被标记为暂不能投递状态，处于该种状态下的消息称为半消息。需要 Producer 对消息的二次确认后，Consumer 才能去消费它）给 Brock 端，消息中携带 B 服务 即将要+100 元的信息。
- 2、当 A 服务知道 Half Message 发送成功后，那么开始第 3 步执行本地事务。
- 3、执行本地事务(会有三种情况 1、执行成功。2、执行失败。3、网络等原因导致没有响应)
- 4.1)、如果本地事务成功，那么 Product 像 Brock 服务器发送 Commit,这样 B 服务就可以消费该 message。
- 4.2)、如果本地事务失败，那么 Product 像 Brock 服务器发送 Rollback,那么就会直接删除上面这条半消息。
- 4.3)、如果因为网络等原因迟迟没有返回失败还是成功，那么会执行 RocketMQ 的回调接口，来进行事务的回查。

从上面流程可以得知 只有 A 服务本地事务执行成功 ，B 服务才能消费该 message。

那么 A 账户减 100 （成功），B 账户加 100 （失败），这时候 B 服务失败怎么办？

如果 B 最终执行失败，几乎可以断定就是代码有问题所以才引起的异常，因为消费端 RocketMQ 有重试机制，如果不是代码问题一般重试几次就能成功。

如果是代码的原因引起多次重试失败后，也没有关系，将该异常记录下来，由人工处理，人工兜底处理后，就可以让事务达到最终的一致性。

RocketMQ 消息顺序怎么保证?

消息的有序性是指消息的消费顺序能够严格保存与消息的发送顺序一致。例如，一个订单产生了 3 条消息，分别是订单创建、订单付款和订单完成。在消息消费时，同一条订单要严格按照这个顺序进行消费，否则业务会发生混乱。同时，不同订单之间的消息又是可以并发消费的，比如可以先执行第三个订单的付款，再执行第二个订单的创建。

RocketMQ 采用了局部顺序一致性的机制，实现了单个队列中的消息严格有序。也就是说，如果想要保证顺序消费，必须将一组消息发送到同一个队列中，然后再由消费者进行注意消费。

RocketMQ 推荐的顺序消费解决方案是：安装业务划分不同的队列，然后将需要顺序消费的消息发往同一队列中即可，不同业务之间的消息仍采用并发消费。这种方式在满足顺序消费的同时提高了消息的处理速度，在一定程度上避免了消息堆积问题

RocketMQ 顺序消息的原理是：

- 在 Producer（生产者） 把一批需要保证顺序的消息发送到同一个 MessageQueue
- Consumer（消费者） 则通过加锁的机制来保证消息消费的顺序性，Broker 端通过对 MessageQueue 进行加锁，保证同一个 MessageQueue 只能被同一个 Consumer 进行消费。

RocketMQ 怎么保证消息不被重复消费?

在业务逻辑中实现**幂等性**，确保即使消息被重复消费，也不会影响业务状态。

例如，对于支付或转账类操作，可以使用唯一订单号或事务 ID 作为幂等性的标识符，确保同样的操作只会被执行一次。

RocketMQ 消息积压了，怎么办?

处理 RocketMQ 的消息积压，其实有点像「大禹治水」，总体思路可以分为**「紧急疏通止损」和「事后排查根因」**两步走。

如果在生产上突然告警，发现有几百万条消息堆积了，我们通常会按以下几个层次来应对：

第一招，也是最本能的反应：尝试给消费者（Consumer）扩容。 既然处理慢了，那直接加几台消费者服务器不就行了吗？ **但这里有一个非常核心、且经常让人踩坑的 RocketMQ 知识点：**在 RocketMQ 里，一个队列（MessageQueue）同一时刻只能被一个消费者机器全权占有。 比如，你这个 Topic 默认只有 4 个队列，那你当初部署了 4 台消费机器，速度已经达到上限了。这时候就算你紧急再加 10 台机器进去，这 10 台机器也会在那儿干瞪眼，根本分不到队列，也就起不到任何加速排队的作用。

那怎么办呢？这就引出我们的第二招，遇到海量积压时真正的「大招」：临时 Topic 嫁接术。 如果积压实在太多，为了快速恢复正常业务，我们会立刻写一套非常简单的「搬砖程序」上线。这个程序没有任何复杂的业务逻辑，它唯一的任务，就是全速把积压的老 Topic 里的消息拉出来，原封不动地发到一个临时建立的**新 Topic** 里去。 绝妙的地方在于，建新 Topic 的时候，我们可以一口气给它分配 **数十或上百个队列**。接着，我们再部署几十台真正干活的业务消费者机器，去对接这个多队列的新 Topic。这样就能突破原来老队列数量的物理限制，以十倍甚至几十倍的速度把积压的账给清掉。等积压处理完了，再切回原来的老 Topic 架构。

第三招，业务兜底方案。 如果跟业务方沟通后，发现系统已经濒临崩溃，而且这些积压的消息（比如一些用户的非关键点击日志、普通的埋点数据）丢了也不影响核心营收。那也可以直接在 RocketMQ 的控制台上，一键把消费组的「消费点位（Offset）」重置到最新的时间点。这就相当于人为地把之前积压的消息全部跳过作废了，先保住当下的系统可用性。

最后，当积压处理完了，一定要去做「事后诸葛亮」排查根因。 消息为什么会积压？十有八九不是因为前面发得太快，而是因为后面消费得太慢了。比如你的消费者代码里是不是在调第三方的接口，恰好那个接口今天网络抖动一直在超时连累了你？又或者是消费者在写 MySQL 的时候，出现了慢查询甚至死锁？只有找到代码里卡脖子的地方，优化掉它（比如引入缓存、优化 SQL），才能保证以后不再发洪水。

总结一下我的回答： 面对 RocketMQ 消息积压，首先要评估积压量，少量积压通过扩容 Consumer（前提是队列数足够干活）来解决；海量积压则使用「转存临时扩容 Topic」的杀手锏进行极致加速；当然业务允许的话跳过消费也是止血捷径；最后再回过头来死磕消费者代码里的性能瓶颈。这是我们排查这类问题的一套比较连贯的组合拳。

kafka

对 Kafka 有什么了解吗?

Kafka 特点如下：

- 高吞吐量、低延迟：kafka 每秒可以处理几十万条消息，它的延迟最低只有几毫秒，每个 topic 可以分多个 partition, consumer group 对 partition 进行 consume 操作。
- 可扩展性：kafka 集群支持热扩展
- 持久性、可靠性：消息被持久化到本地磁盘，并且支持数据备份防止数据丢失
- 容错性：允许集群中节点失败（若副本数量为 n,则允许 n-1 个节点失败）
- 高并发：支持数千个客户端同时读写

Kafka 为什么这么快?

- **顺序写入优化**：Kafka 将消息顺序写入磁盘，减少了磁盘的寻道时间。这种方式比随机写入更高效，因为磁盘读写头在顺序写入时只需移动一次。
- **批量处理技术**：Kafka 支持批量发送消息，这意味着生产者在发送消息时可以等待直到有足够的数据积累到一定量，然后再发送。这种方法减少了网络开销和磁盘 I/O 操作的次数，从而提高了吞吐量。
- **零拷贝技术**：Kafka 使用零拷贝技术，可以直接将数据从磁盘发送到网络套接字，避免了在用户空间和内核空间之间的多次数据拷贝。这大幅降低了 CPU 和内存的负载，提高了数据传输效率。
- **压缩技术**：Kafka 支持对消息进行压缩，这不仅减少了网络传输的数据量，还提高了整体的吞吐量。

kafka 的模型介绍一下，kafka 是推送还是拉取?

> 消费者模型

消息由生产者发送到 kafka 集群后，会被消费者消费。一般来说我们的消费模型有两种：推送模型(psuh)和拉取模型(pull)。

推送模型（push）

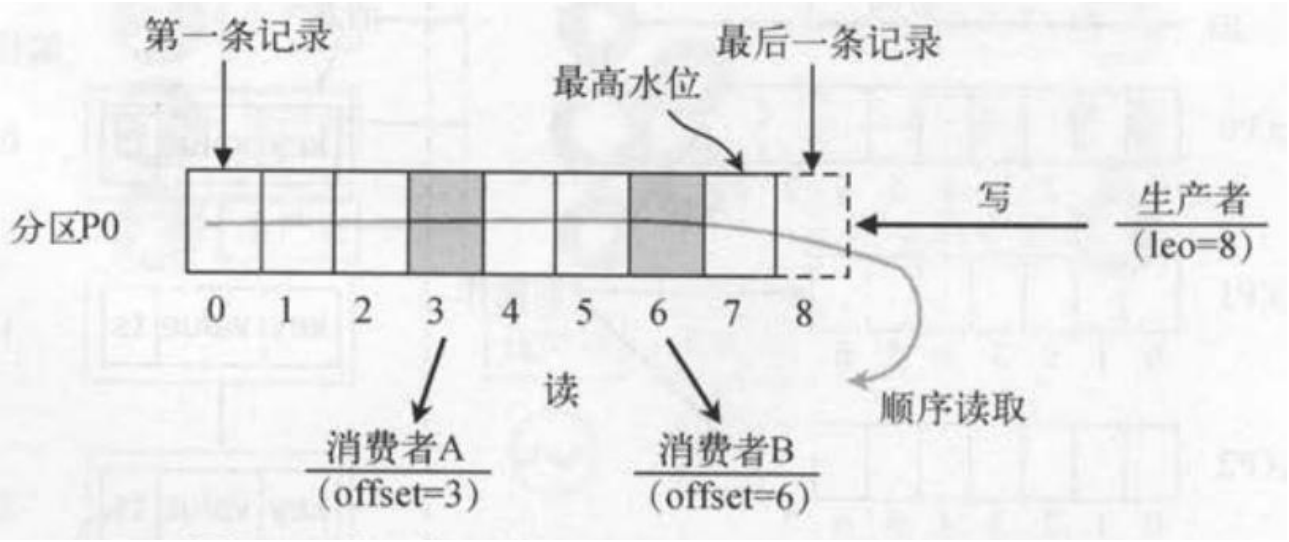
- 基于推送模型（push）的消息系统，有消息代理记录消费者的消费状态。
- 消息代理在将消息推送到消费者后，标记这条消息已经消费，但这种方式无法很好地保证消费被处理。
- 如果要保证消息被处理，消息代理发送完消息后，要设置状态为「已发送」，只要收到消费者的确认请求后才更新为「已消费」，这就需要代理中记录所有的消费状态，但显然这种方式不可取。

缺点：

- **push 模式很难适应消费速率不同的消费者**
- 因为消息发送速率是由 broker 决定的，push 模式的目标是尽可能以最快速度传递消息，但是这样很容易造成 consumer 来不及处理消息，典型的表现就是拒绝服务以及网络拥塞。

拉取模型(pull)

kafka 采用拉取模型，由消费者自己记录消费状态，每个消费者互相独立地顺序拉取每个分区的消息。

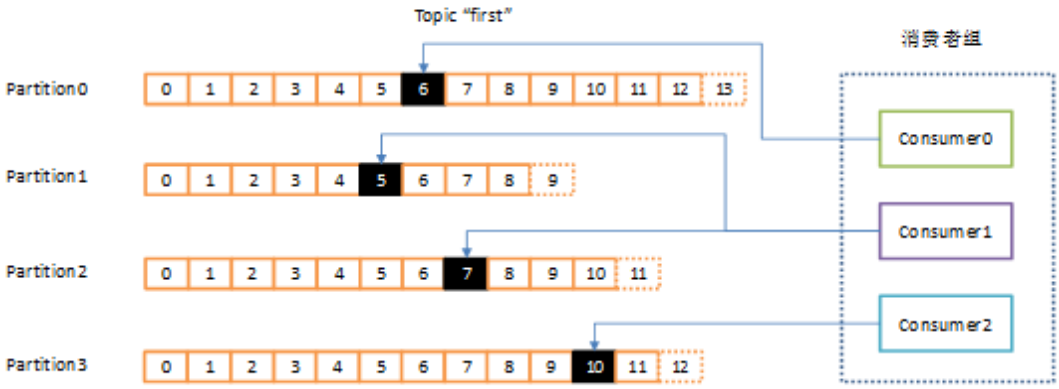


说明：

- 有两个消费者（不同消费者组）拉取同一个主题的消息，消费者 A 的消费进度是 3，消费者 B 的消费进度是 6。
- 消费者拉取的最大上限通过最高水位（watermark）控制，生产者最新写入的消息如果还没有达到备份数量，对消费者是不可见的。
- 这种由消费者控制偏移量的优点是：**消费者可以按照任意的顺序消费消息**。比如，消费者可以重置到旧的偏移量，重新处理之前已经消费过的消息；或者直接跳到最近的位置，从当前的时刻开始消费。

消费者组

kafka 消费者是以 consumer group 消费者组的方式工作，由一个或者多个消费者组成一个组，共同消费一个 topic。每个分区在同一时间只能由 group 中的一个消费者读取，但是多个 group 可以同时消费这个 partition。



上图中，有一个由三个消费者组成的 group，有一个消费者读取主题中的两个分区，另外两个分别读取一个分区。某个消费者读取某个分区，也可以叫做某个消费者是某个分区的拥有者。

优点在于：

- 消费者可以通过水平扩展的方式同时读取大量的消息。
- 如果一个消费者失败了，那么其他的 group 成员会自动负载均衡读取之前失败的消费者读取的分区。

> 消费方式

kafka 消费者采用 pull（拉）模式从 broker 中读取数据。

pull 的优点：

- pull 模式可以根据 consumer 的消费能力以适当的速率消费消息

缺点：

- 如果 kafka 没有数据，消费者可能会陷入循环中，一直返回空数据。针对这一点，Kafka 的消费者在消费数据时会传入一个时长参数 timeout，如果当前没有数据可供消费，consumer 会等待一段时间之后再返回，这段时长即为 timeout。

Kafka 如何保证顺序读取消息?

Kafka 可以保证在同一个分区内消息是有序的，生产者写入到同一分区的消息会按照写入顺序追加到分区日志文件中，消费者从分区中读取消息时也会按照这个顺序。这是 Kafka 天然具备的特性。

要在 Kafka 中保证顺序读取消息，需要结合生产者、消费者的配置以及合适的业务处理逻辑来实现。以下具体说明如何实现顺序读取消息：

- 生产者端确保消息顺序：为了保证消息写入同一分区从而确保顺序性，生产者需要将消息发送到指定分区。可以通过自定义分区器来实现，通过为消息指定相同的 Key，保证相同 Key 的消息发送到同一分区。
- 消费者端保证顺序消费：消费者在消费消息时，需要单线程消费同一分区的消息，这样才能保证按顺序处理消息。如果使用多线程消费同一分区，就无法保证消息处理的顺序性。

Kafka 本身不能保证跨分区的消息顺序性，如果需要全局的消息顺序性，通常有以下两种方法：

- 只使用一个分区：将所有消息都写入到同一个分区，消费者也只从这个分区消费消息。但这种方式会导致 Kafka 的并行处理能力下降，因为 Kafka 的性能优势在于多分区并行处理。
- 业务层面保证：在业务代码中对消息进行编号或添加时间戳等标识，消费者在消费消息后，根据这些标识对消息进行排序处理。但这种方式会增加业务代码的复杂度。

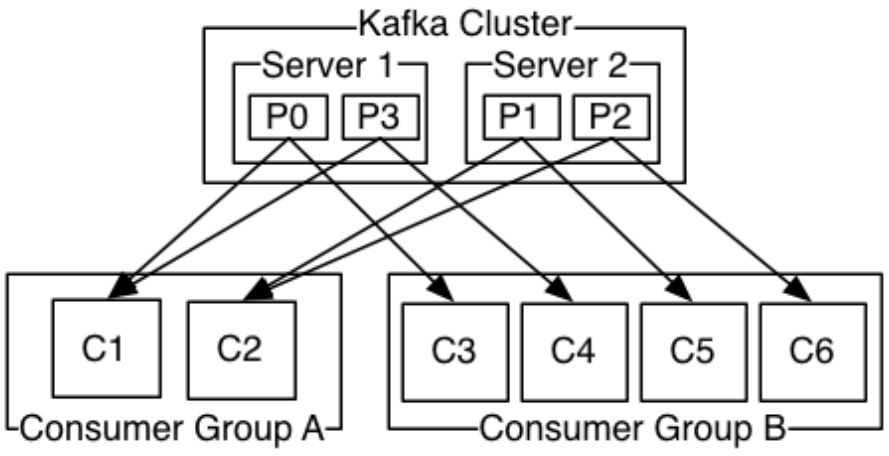
kafka 消息积压怎么办?

Kafka 消息积压是一个常见的问题，它可能会导致数据处理延迟，甚至影响业务的正常运行，下面是一些解决 Kafka 消息积压问题的常用方法：

- 增加消费者实例可以提高消息的消费速度，从而缓解积压问题。你需要确保消费者组中的消费者数量不超过分区数量，因为一个分区同一时间只能被一个消费者消费。
- 增加 Kafka 主题的分区数量可以提高消息的并行处理能力。在创建新分区后，你需要重新平衡消费者组，让更多的消费者可以同时消费消息。

Kafka 为什么一个分区只能由消费者组的一个消费者消费？这样设计的意义是什么?

同一时刻，一条消息只能被组中的一个消费者实例消费

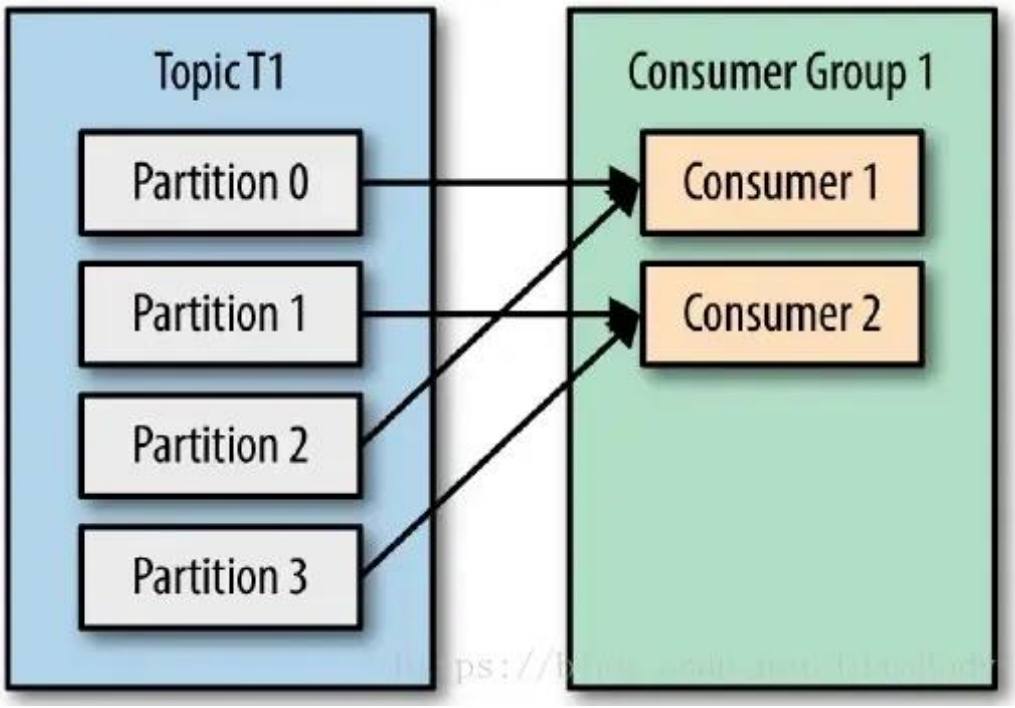


如果两个消费者负责同一个分区，那么就意味着两个消费者同时读取分区的消息，由于消费者自己可以控制读取消息的 offset，就有可能 C1 才读到 2，而 C1 读到 1，C1 还没处理完，C2 已经读到 3

了，则会造成很多浪费，因为这就相当于多线程读取同一个消息，会造成消息处理的重复，且不能保证消息的顺序。

如果有一个消费主题 topic，有一个消费组 group，topic 有 10 个分区，消费线程数和分区数的关系是怎么样的?

topic 下的一个分区只能被同一个 consumer group 下的一个 consumer 线程来消费，但反之并不成立，即一个 consumer 线程可以消费多个分区的数据，比如 Kafka 提供的 ConsoleConsumer，默认就只是一个线程来消费所有分区的数据。



所以，**分区数决定了同组消费者个数的上限**。

如果你的分区数是 N，那么最好线程数也保持为 N，这样通常能够达到最大的吞吐量。超过 N 的配置只是浪费系统资源，因为多出的线程不会被分配到任何分区。

消息中间件如何做到高可用?

关于消息中间件如何做到高可用，其实无论我们用的是 Kafka、RocketMQ 还是 RabbitMQ，它们底层实现高可用的「核心心法」都是相通的。

如果用大白话来概括，消息中间件的高可用其实就是要保障两件事：**第一是「机器挂了服务不能停」，第二是「机器挂了数据不能丢」。**

为了做到这两点，业界通常会组合采用三套关键机制，我给你梳理一下：

首先第一步，也是最基础的：打破单点，采用「集群与多副本机制」。 单机总有罢工的时候，所以咱们必须得上物理集群。但光有集群还不够，如果一条消息只存在 A 机器上，A 机器一断电，消息还是没法消费。所以核心在于**「数据要有分身」，也就是要有主从（Master-Slave）或者副本（Replica）。 当生产者把消息发给「主节点」后，系统会在后台悄悄把这条消息复制到「从节点」上去。 *这里通常会有个经典的面试考点，就是怎么复制？* 如果是**「异步复制」，主节点收到消息直接告诉开发者成功了，然后再慢慢同步给从节点，这样速度最快，但如果主节点突然宕机，没来得及同步的数据就丢了。 如果是**「同步复制」**，主节点必须等到从节点也写进磁盘了，才告诉开发者成功。这样极其安全，但性能会受影响。所以在实际生产中，我们通常会根据业务的敏感程度来权衡，找一个合适的折中点。

第二步，有了数据副本之后，还得有「自动选主（故障转移）机制」。 打个比方，即使从节点（Slave）手里有完整的数据，但如果主节点（Master）宕机了，从节点不知道自己该转正，系统照样会瘫痪。 所以，高可用 MQ 必须有一个「最高指挥部」（比如 Kafka 依赖 Zookeeper/KRaft，RocketMQ 使用底层的 Dledger 机制）。它们的底层会跑着一套类似 Raft 的选举算法，大家时刻保持心跳互相拉扯。一旦大家发现主节点不跳了，就会马上发起「民主投票」，在剩下的从节点里，挑一个数据最全、最健康的从节点，火速提拔为新的主节点，立刻接管写入请求。整个过程往往在秒级甚至毫秒级自动完成，业务层大多数时候甚至感觉不到停顿。

第三步，最外面还需要一层「动态路由感知机制」。 现在我们的 MQ 服务端已经能在宕机时自动切换了，那写代码的「生产者」和「消费者」怎么知道该连哪台新机器呢？ 这就需要类似 RocketMQ 的 NameServer 或者 Kafka 的高可用客户端出场了。这些协调组件时刻盯着 Broker 们的一举一动，一旦发生了上面说的主从切换，它们会迅速把最新的路由表（也就是谁是新老大）下发给业务端的代码。业务代码拿到了最新的地址，就能瞬间调整方向，继续给正常存活的节点发消息。

总结一下： 消息中间件的高可用，本质上就是依靠**「多节点副本机制」保证了数据不孤单，依靠「自动选举与故障转移」保证了服务端大脑不宕机，最后依靠**「动态路由协调组件」**无缝衔接了客户端。这三套组合拳打下来，才真正构筑了企业级消息系统极其坚固的高可用基石。

Kafka 和 RocketMQ 消息确认机制有什么不同?

关于 Kafka 和 RocketMQ 在消息确认机制（Ack 机制）上的区别，这其实跟它们两者的诞生背景有很大关系。Kafka 骨子里是为了海量日志处理诞生的，追求的是**极致的吞吐量**；而 RocketMQ 是阿里为了支撑双十一电商业务量身打造的，追求的是**极其严苛的业务可靠性**。

我们可以从「生产者发消息的确认」和「消费者消费后的确认」两个阶段来看，由于侧重点不同，其实差别挺大的。

第一，我们先看「生产者端」的确认机制（也就是 MQ 告诉大盘：这条消息我收妥了）。

- Kafka 玩的是基于副本的 `acks` 机制。它给开发者留了三个选项：`acks=0`（发出去就不管了）、`acks=1`（只要 Leader 节点存下来就返回成功）、`acks=all/-1`（必须等所有同步副本 ISR 都存下来才算成功）。它的核心设计思路是围绕着**分布式集群的副本同步机制**来确保存储可靠性的。
- RocketMQ 的侧重点有点不一样，它更关注单机层面和主从架构上的物理落盘。它的确认机制主要依赖于你的集群配置：是**同步刷盘还是异步刷盘**？是**同步复制（主从都写完才返回）还是异步复制**？相对于 Kafka，RocketMQ 在业务上往往会配置成「同步双写」，也就是追求物理级别的不丢消息。

第二，也是我们在日常写代码时感知最深的区别，也就是「消费者端」的确认机制。 用一句通俗的话来说：**Kafka 是「进度条模式」，而 RocketMQ 是「单据签收模式」。**

- **Kafka 的消费者确认，叫作「提交偏移量（Offset Commit）」。** 简单说，Kafka 的队列就像一本书，消费者消费就像在看书，每次告诉 Kafka 的是「我已经看到第 100 页了」。这就带来一个问题：如果是批量消费，第 99 条消息处理失败了，第 100 条成功了。你要是提交了第 100 页的进度，第 99 条就相当于被跳过丢失了；你要是不提交，下次重启又得从头再看一遍。Kafka 原生**没有**很好地提供单条消息级别的失败重试机制，通常需要我们在业务代码里自己捕获异常去处理。
- **RocketMQ 的消费者确认，是非常贴合业务逻辑的。** 它虽然底层也有进度，但在代码层面，它要求消费者针对**每一条/每一批消息**明确返回状态：要么是 `CONSUME_SUCCESS`（消费成功），要么是 `RECONSUME_LATER`（稍后重试）。 绝妙的地方在于，如果你返回了失败或者抛了异常，RocketMQ 不会卡在那里，它会自动把这条消息塞进一个内置的**「重试队列」**里，**并且有一套阶梯式的延迟等待时间（比如等 10 秒、30 秒、1 分钟后再让你消费）；如果重试了十几次都不行，最后它还会帮你把消息放进**「死信队列（DLQ）」**，方便人工顺藤摸瓜去排查。这种保姆式的单条消息确认和重试机制，对电商等复杂的业务场景极其友好。

总结一下： Kafka 的确认机制强调整体批次和吞吐，更注重副本之间的同步，消费者端采用指针偏移量的方式，稍微有点粗粒度；而 RocketMQ 的确认机制带有极强的「业务基因」，不仅关注物理落盘，在消费者端更是提供了精细到单条消息级别的成功/失败状态确认和自动重试闭环。这也是为什么在大数据领域大家首选 Kafka，而在稍微复杂的微服务业务里，大家更偏爱 RocketMQ 的原因。

Kafka 和 RocketMQ 的 broker 架构有什么区别

Kafka 是为了大数据领域的日志流处理设计的，而 RocketMQ 是阿里当年在双十一海量业务场景下，因为觉得 Kafka 没法完全满足需求，从而借鉴改造出来的。

在 Broker 的核心架构设计上，它们最大的区别主要集中在**「文件存储模型」、「协调机制」以及「高可用粒度」**三个方面，详细说一下：

最致命、也是最核心的区别，在于「文件存储模型」。

- **Kafka 的 Broker 采用的是「独立的分区文件」设计。** 打个比方，Kafka 就像是给每个具体的业务（Partition）单独准备了一个小本子。业务 A 的消息写进 A 的本子，业务 B 的消息写进 B 的本子。当主题（Topic）比较少时，这种设计速度极快，是非常纯粹的磁盘顺序写。 *但问题来了*，在电商微服务场景下，可能成千上万个系统都在用 MQ，如果产生了上万个主题，Kafka 的 Broker 底层就会有上万个文件同时在写入。原本的「顺序写」瞬间退化成了「磁盘随机写」，性能暴跌，机器卡死。
- **RocketMQ 的 Broker 则是「大一统的混合存储」设计。** 阿里就是为了解决上面说的海量 Topic 导致性能下降的问题，设计了著名的 **CommitLog + ConsumeQueue** 架构。 不管你有多少个主题，所有的消息来到 Broker 后，不管三七二十一，全部顺序追加写到一个公共的、巨大的日志文件里（也就是 CommitLog）。这就保证了无论 Topic 数量怎么暴增，写入永远是极致的磁盘顺序写。 *那消费者怎么找自己的消息呢？* Broker 后台会有一波专门的线程，实时把大文件里的消息位置挑出来，整理成一本本轻量级的「目录」（也就是 ConsumeQueue）。消费者顺着目录去大文件里掏数据就行了。这就是两者在底层架构上水火不容的区别。

第二个区别，在于 Broker 身后的「协调大脑」不同。

- **Kafka 极其依赖外部组件（传统架构依赖 Zookeeper，新版换成了内置的 KRaft）。** Kafka 的 Broker 是个纯粹的打工人，它把元数据管理、集群选主这种极其复杂的脑力劳动全交给了 Zookeeper（或者 KRaft 控制器节点）。节点一多，心跳保活和数据同步的开销是非常大的。
- **RocketMQ 的 Broker 搭配的是极其轻量级的 NameServer。** 阿里当年测试时嫌弃 Zookeeper 实在太重、太容易在网络风暴中崩溃，于是自己写了个极简的 NameServer。NameServer 节点之间甚至都是互相不通信的，全靠底下的 Broker 们主动给所有的 NameServer 进行「多头汇报」。这种设计让挂掉任何一个 NameServer 都不影响大局，Broker 架构变得异常简单稳定。

第三个区别，高可用的「粒度」不一样。

- **Kafka 的高可用是「Partition 分区级别」的。** 一台 Broker 宕机了，Kafka 是在底层成百上千个小本子（Partition）里，一个个去重新选举新的 Leader，这需要耗费一定时间。
- **RocketMQ 的高可用是「Broker 服务器节点级别」的。** 它更偏向于传统数据库的主从模式（Master-Slave）。Master 挂了，消费者就直接无缝切换去读 Slave。相对来说，管理层级更高，机制也更偏向于物理机器层面的容灾。

总结一下： Kafka 的 Broker 架构就像是**「分布式的文件系统」，每个分区自治，适合大数据场景下少 Topic 的大吞吐量；而 RocketMQ 的 Broker 是典型的**「数据库引擎思维」**，通过统一的 CommitLog 解决海量 Topic 的写入瓶颈，又用 NameServer 把分布式协调做到极致轻量化，是非常强悍的企业级业务微服务利器。

RabbitMQ

RabbitMQ 和 AMQP 是什么关系?

如果用一句话来简单概括，那就是：**AMQP 是一套理论标准，而 RabbitMQ 是这套标准的具体落地实现。**

如果你写过 Java，那它俩的关系就像是我们代码里的**「接口（Interface）」和「实现类」**的关系。我们可以这样来理解：

首先，AMQP 是「图纸」。 AMQP 的全名叫「高级消息队列协议」。请注意，它是一个「协议」，本身并不是一个可以直接安装运行的软件。它就像一份详细的交通规则或者建筑图纸，规定了一套理想的消息队列系统应该长什么样。比如，它在协议里定义了：消息发出来之后不能直接进队列，中间得有个东西叫「交换机（Exchange）」，还得有路由键来指明消息去哪儿。它只管定规矩，不负责写代码。

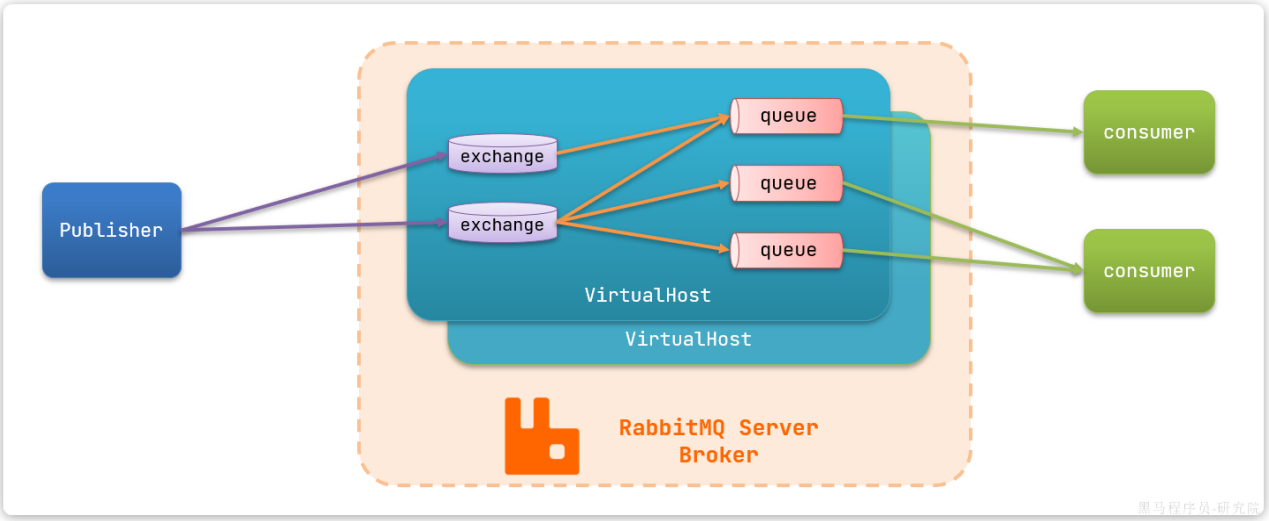
其次，RabbitMQ 是「真房子」。 有了图纸，肯定得有人把它盖出来，RabbitMQ 就是那座盖好的房子。其实当年有很多团队想去完成 AMQP 这套理论，而 RabbitMQ 的开发团队用并发性能极高的 Erlang 语言，完美地照着 AMQP 这套协议标准，写出了一个真正可以通电运行的软件产品。可以说，RabbitMQ 是今天市面上把 AMQP 协议贯彻得最原汁原味、也是最知名的一款消息中间件。

理解这层关系，对我们实际开发的意义在哪里呢？ 就在于**「灵活的路由能力」。很多简单的 MQ 就是生产者发消息、队列存消息、消费者取消息，一条直线而已。但在 AMQP 协议的规范里，事情没那么简单。受 AMQP 基因的影响，我们在用 RabbitMQ 的时候，生产者发的消息实际上是先发给了**交换机（Exchange）**，交换机就像一个拥有极其丰富经验的「邮局分拣员」，它可以根据我们配置的各种规则——比如按名字直接投递、按通配符模式投递、或者干脆直接广播打包，把一条消

息精准地分发给不同的目标队列。这种非常强大的路由机制，恰恰就是 AMQP 协议赋予 RabbitMQ 最大的一张王牌。

总结一下就是： AMQP 提供了消息队列应该怎么设计的核心灵魂和规范，而 RabbitMQ 把这套规范变成了现实，成为了一款企业级的高性能消息中间件。这就是它俩的关系。

RabbitMQ 核心组件有哪些?



RabbitMQ 的**核心组件**，都是消息从发出到消费必不可少的部分：

- 首先是**生产者（Producer）**，它就是发送消息的一方，比如我们的业务服务，负责把需要异步处理的消息，交给 RabbitMQ，不会直接去找消费者。
- 然后是**消费者（Consumer）**，它是监听并处理消息的一方，一直盯着队列，一旦队列里有新消息，就会取出来执行业务逻辑。
- 接下来是**队列（Queue）**，这是 RabbitMQ 里**真正存储消息**的地方，采用先进先出的机制，消息会一直存在这里，直到被消费者取走，是整个消息队列的核心载体。
- 然后是**交换机（Exchange）**，生产者不会直接把消息发给队列，而是先发给交换机，它相当于一个**消息分发器**，会按照我们设定的规则，把消息路由到对应的队列里，也就是我们刚才说的四种交换机类型。
- 还有**路由键（RoutingKey）** 和**绑定（Binding）**，绑定是把交换机和队列关联起来的纽带，路由键则是生产者发消息时带的标识，交换机会根据路由键和绑定的规则，判断消息该投到哪个队列。

- 最后是**连接（Connection）** 和**信道（Channel）**，Connection 是客户端和 RabbitMQ 之间的 TCP 长连接，而 Channel 是建立在连接之上的轻量级信道，因为 TCP 连接创建销毁开销很大，所以用多个信道来传输消息，能大大节省资源。

简单总结就是：**生产者发消息给交换机，交换机通过路由键和绑定，把消息投到队列，最后消费者从队列取消息消费**，整个流程就靠这些核心组件完成。

RabbitMQ 有哪几种交换机类型?

RabbitMQ 里**核心的交换机一共四种**，分别是直连、扇形、主题和头交换机，其中前三种是工作中最常用的，头交换机基本很少用到。

- 第一种是**直连交换机（Direct）**，它是**精准匹配**的模式，消息会根据**路由键完全一致**，才转发到对应的队列。比如路由键设成 `user.login`，就只会发给绑定了这个 exact 键的队列，适合一对一、精准投递的场景。
- 第二种是**扇形交换机（Fanout）**，它就是**广播模式**，完全不看路由键，只要队列绑定了这个交换机，所有消息都会**无脑发给所有队列**。适合群发通知、多服务同步更新这类场景，效率很高。
- 第三种是**主题交换机（Topic）**，它支持**路由键模糊匹配**，用 `*` 匹配一个单词、`#` 匹配多个或 0 个单词，是最灵活的一种。比如用 `order.#`，就能匹配 `order.create`、`order.pay` 所有以 order 开头的路由键，适合按业务主题分类、批量路由的场景。
- 第四种是**头交换机（Headers）**，它不看路由键，而是根据**消息头里的键值对**来匹配，但是性能比较差，实际开发里几乎不用，简单了解就行。

RabbitMQ 的特性你知道哪些?

RabbitMQ 以 **可靠性**、**灵活性** 和 **易扩展性** 为核心优势，适合需要稳定消息传递的复杂系统。其丰富的插件和协议支持使其在微服务、IoT、金融等领域广泛应用，比较核心的特性有如下：

- **持久化机制**：RabbitMQ 支持消息、队列和交换器的持久化。当启用持久化时，消息会被写入磁盘，即使 RabbitMQ 服务器重启，消息也不会丢失。例如，在声明队列时可以设置 `durable` 参数为 `true` 来实现队列的持久化：

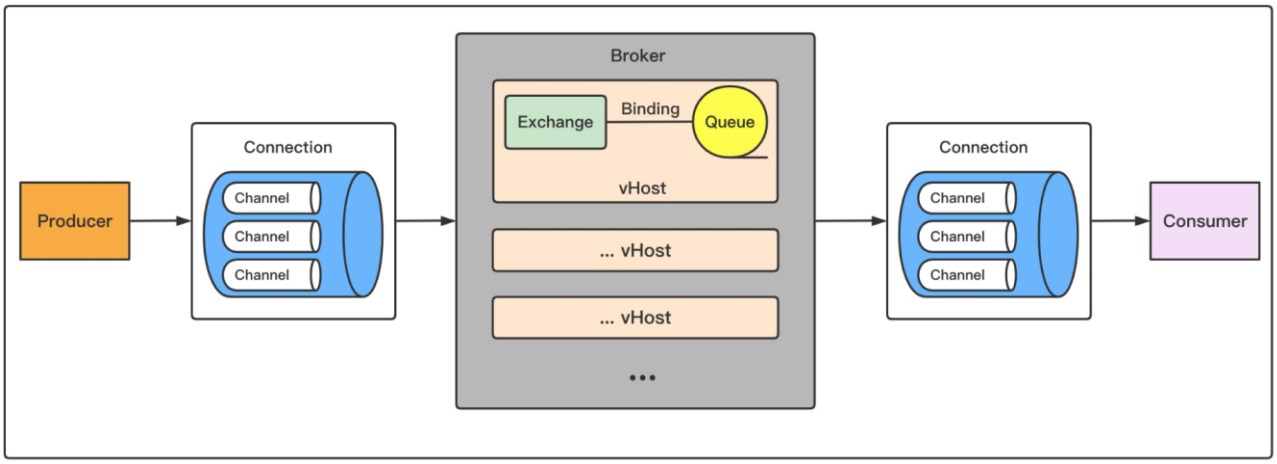
```
import pika


connection = pika.BlockingConnection(pika.ConnectionParameters('localhost'))
channel = connection.channel()
```

```
# 声明一个持久化队列
channel.queue_declare(queue='durable_queue', durable=True)
```

- **消息确认机制**：提供了生产者确认和消费者确认机制。生产者可以设置 `confirm` 模式，当消息成功到达 RabbitMQ 服务器时，会收到确认消息；消费者在处理完消息后，可以向 RabbitMQ 发送确认信号，告知服务器该消息已被成功处理，服务器才会将消息从队列中删除。
- **镜像队列**：支持创建镜像队列，将队列的内容复制到多个节点上，提高消息的可用性和可靠性。当一个节点出现故障时，其他节点仍然可以提供服务，确保消息不会丢失。
- **多种交换器类型**：RabbitMQ 提供了多种类型的交换器，如直连交换器（Direct Exchange）、扇形交换器（Fanout Exchange）、主题交换器（Topic Exchange）和头部交换器（Headers Exchange）。不同类型的交换器根据不同的规则将消息路由到队列中。例如，扇形交换器会将接收到的消息广播到所有绑定的队列中；主题交换器则根据消息的路由键和绑定键的匹配规则进行路由。

RabbitMQ 的底层架构是什么?



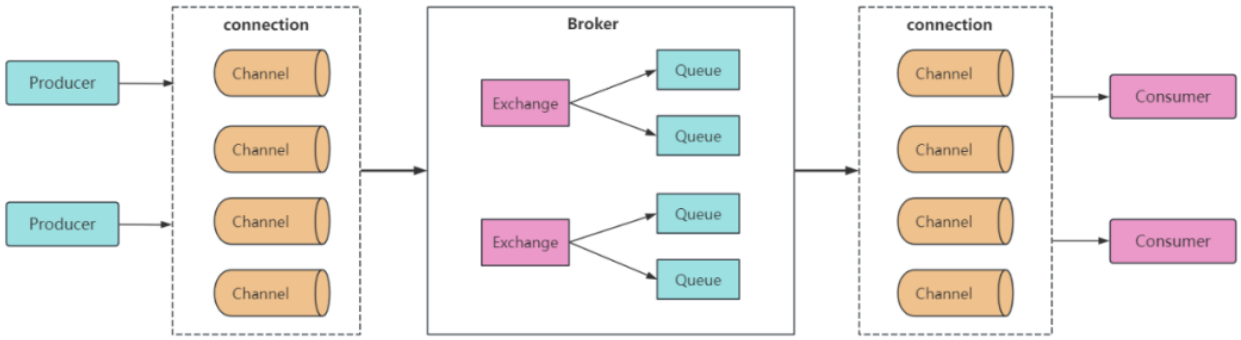
以下是 RabbitMQ 的一些核心架构组件和特性：

- **核心组件**：生产者负责发送消息到 RabbitMQ、消费者负责从 RabbitMQ 接收并处理消息、RabbitMQ 本身负责存储和转发消息。
- **交换机**：交换机接收来自生产者的消息，并根据 routing key 和绑定规则将消息路由到一个或多个队列。
- **持久化**：RabbitMQ 支持消息的持久化，可以将消息保存在磁盘上，以确保在 RabbitMQ 重启后消息不丢失，队列也可以设置为持久化，以保证其结构在重启后不会丢失。

- **确认机制**：为了确保消息可靠送达，RabbitMQ 使用确认机制，费者在处理完消息后发送确认给 RabbitMQ，未确认的消息会重新入队。
- **高可用性**：RabbitMQ 提供了集群模式，可以将多个 RabbitMQ 实例组成一个集群，以提高可用性和负载均衡。通过镜像队列，可以在多个节点上复制同一队列的内容，以防止单点故障。

消息中间件 rabbitMQ 的可靠性保障怎么做?

RabbitMQ 的可靠性保障核心是确保 「消息不丢失、不重复、不积压」，这需要从生产者、RabbitMQ 服务器、消费者三个环节层层把关。



首先是**生产者环节**，要保证消息能成功投递到交换机。这就需要开启生产者确认机制（Publisher Confirm），一旦消息被交换机接收，RabbitMQ 会返回确认通知，若投递失败（比如交换机不存在），生产者能及时重试；同时建议开启事务机制，但事务会影响吞吐量，所以大部分场景下用确认机制更合适。另外，生产者要处理网络异常、连接中断等问题，比如设置合理的重试次数和间隔，避免因临时故障导致消息丢失。

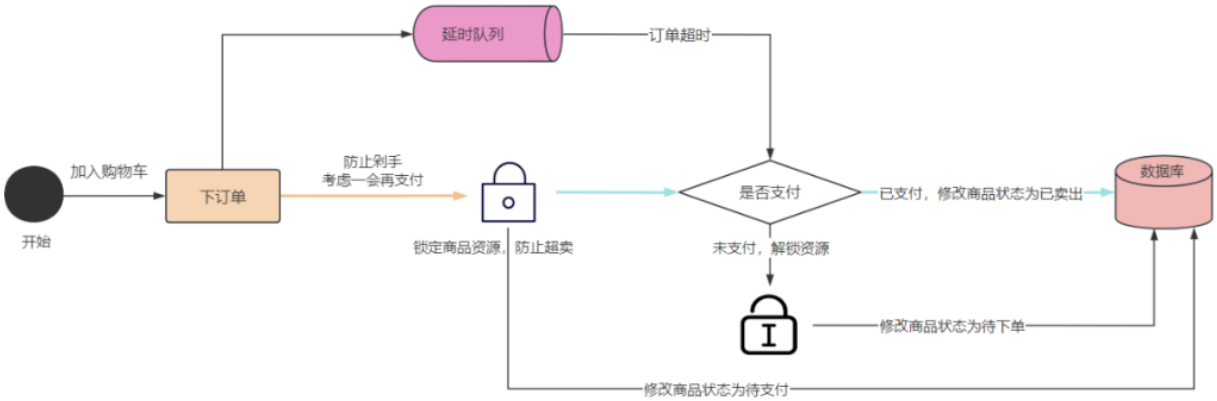
然后是 **RabbitMQ 服务器环节**，要防止消息在服务器端丢失。这就需要给队列和消息都设置持久化：队列持久化能保证 RabbitMQ 重启后队列不消失，消息持久化则能保证消息在服务器重启后不丢失（通过将消息写入磁盘实现）。同时，要合理设置交换机和队列的绑定关系，避免因绑定错误导致消息无法路由到队列（可以开启交换机的死信功能，将无法路由的消息转发到死信队列，后续人工处理）。另外，服务器要做好集群部署和数据备份，比如主从复制、镜像队列，防止单点故障导致消息丢失。

最后是**消费者环节**，要保证消息能被正确处理且不重复消费。消费者需要开启手动确认模式（ack），只有当消息被成功处理（比如业务逻辑执行完成、数据入库成功）后，才手动发送 ack 通知 RabbitMQ 删除消息；若处理失败（比如业务异常、系统崩溃），则不发送 ack，RabbitMQ 会将消息重新入队，等待后续重试。同时，要处理消息重复消费的问题，这就需要在业务层面做幂等性设计，比如给消息设置唯一 ID，消费时先查询该 ID 是否已处理，若已处理则直接忽略，避免重

复执行业务逻辑。另外，消费者要控制消费速度，避免因消费过慢导致消息积压，可以通过限流机制（比如每次只获取 10 条消息，处理完再获取下一批）来平衡消费能力和消息堆积问题。

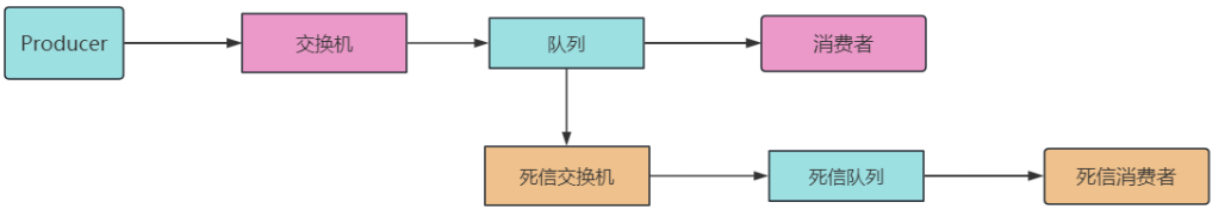
讲一下 rabbitMQ 的延迟队列和死信机制

先说说**延迟队列**，它的核心就是让消息不是发出去就被消费，而是等指定时间后才让消费者处理。



比如电商里订单创建后 30 分钟没支付要自动取消，就不用搞个定时任务一直查数据库，直接用延迟队列就行。具体就是生产者发消息的时候，给消息设置一个延迟时间，这个消息会先到一个专门的延迟交换机（一般是 x-delayed-message 类型），交换机不会马上转发，而是等够了设置的时间，再把消息转到真正的业务队列，消费者监听这个业务队列，到点就收到消息执行取消订单的逻辑，特别省心。

再看**死信机制**，死信就是那些没法正常处理的消息，相当于给这些 「问题消息」 找个兜底的地方，不让它们一直堆在业务队列里占资源。



那什么样的消息会变成死信呢？比如消费者处理消息时出了问题，明确拒绝接收而且不让它重新回到原队列；或者消息在队列里放太久过期了（队列设置了存活时间）；还有队列满了，新消息进不来，最老的那些消息就会被挤成死信。这时候我们给业务队列提前配置好死信交换机和对应的死信队列，一旦有消息变成死信，RabbitMQ 就会自动把它转到死信队列里，之后我们可以专门监听死信队列，要么记录日志找问题，要么人工处理，或者设置重试机制，不会让问题消息石沉大海。

实际项目里这俩经常一起用，比如订单取消场景，我们可以给订单队列配置死信交换机和死信队列，生产者通过延迟队列给消息设置 30 分钟延迟。如果 30 分钟内用户支付了，消费者处理成功

就正常确认；要是没支付，消息在订单队列里过期就变成死信，自动转到死信队列，死信队列的消费者收到后就执行取消订单、释放库存的操作，同时记录日志，这样既实现了延迟处理，又保证了就算中间出问题，消息也有兜底，流程不会断。

简单总结下，延迟队列管 「消息什么时候处理」，死信机制管 「消息处理失败或过期了怎么办」。
